


# Politecnico di Milano

Anno accademico 2012-2013

## Ingegneria del Software – Appello del 29 Giugno 2012

| | | |
|---|---|---|
| Cognome: | | |
| Nome: | | Matricola: |
| Sezione (segnarne una): ☐ Baresi | ☐ Ghezzi | ☐ San Pietro |

### Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h.

**Esercizio 1:**

**Esercizio 2:**

**Esercizio 3:**

**Esercizio 4:**

# Esercizio 1

Si consideri un'applicazione che gestisce archivi di film. Un film può essere a colori o in bianco e nero, e può essere muto. In ogni caso, ha un titolo, un anno di pubblicazione, e un elenco di artisti, fra cui il regista e un cast di attori.

Il sistema deve permettere ai soli utenti registrati di inserire proprie recensioni, relative a un film o a un artista per un certo film. L'utente quindi seleziona innanzitutto il film e quindi inserisce direttamente la recensione (se si tratta di recensione del film) oppure cerca anche l'artista e quindi ne inserisce la recensione.

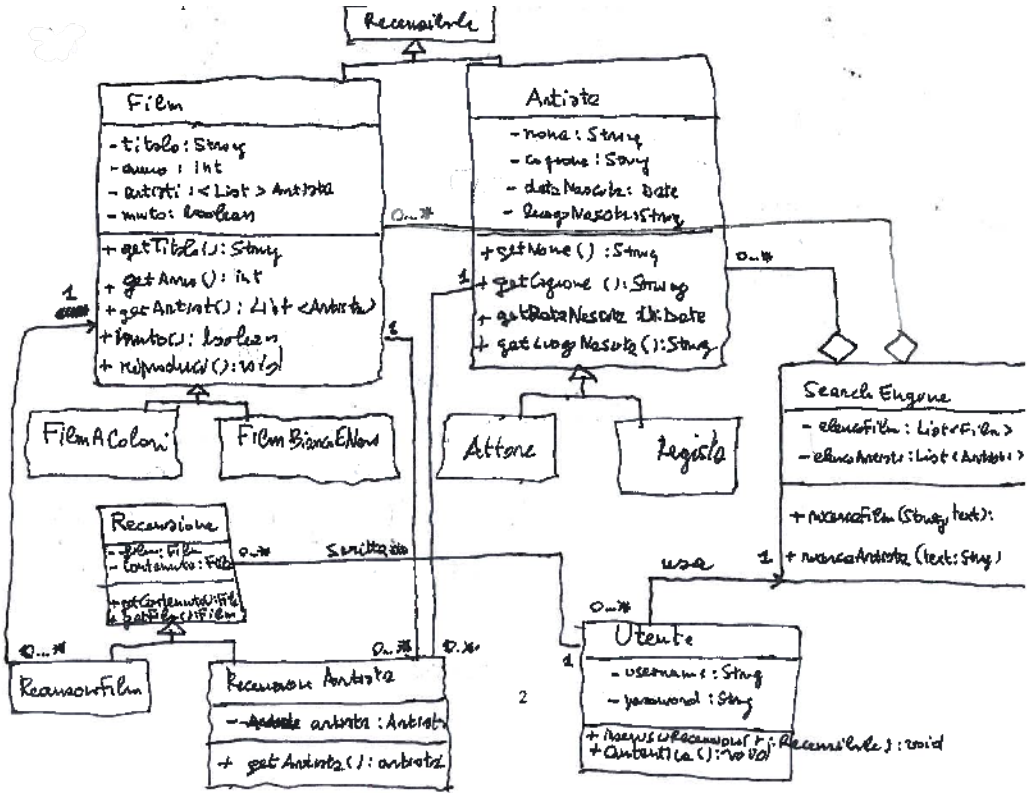
Si chiede di definire:

1. il diagramma UML delle classi che rappresenta questa specifica, arricchendo eventualmente la descrizione con dettagli che qui non sono specificati (ma senza esagerare!);

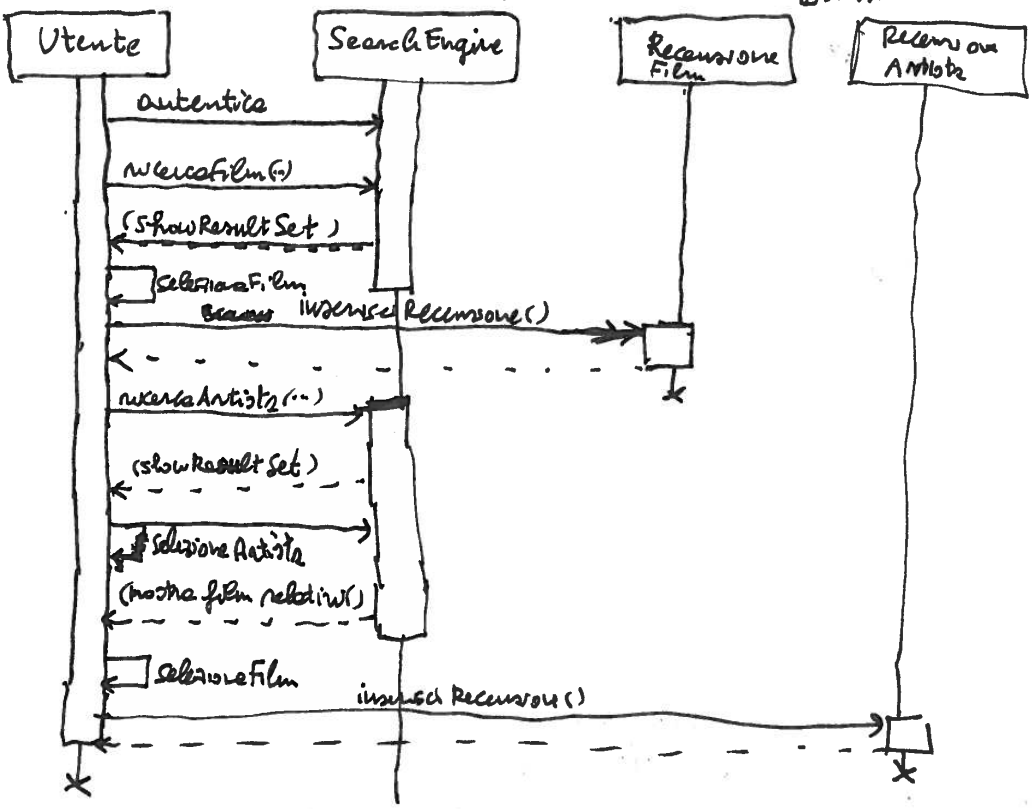
2. la specifica dell'aggiunta di una recensione, con relativa autenticazione, mediante un sequence diagram di UML;

3. la definizione dell'utilizzo di un film mediante un diagramma a stati gerarchico. Per poter guardare il film, occorre innanzitutto caricarlo (operazione LOAD). Il film caricato è predisposto per la visione a partire dall'inizio. Per vedere il film, occorre fornire il comando START. Il comando PAUSE ferma la visione a un certo punto e la ripresa avviene mediante il comando START. Il comando STOP ferma la visione, riposizionando il sistema all'inizio del film. Il comando RESTART consente di riprendere la visione dall'inizio: esso corrisponde dunque all'abbreviazione della sequenza: STOP START (purché ovviamente il film sia stato caricato). Infine, l'operazione END scarica il film dalla memoria. Si ipotizzi anche che un macrostato PLAYABLE indichi che il film è stato caricato in memoria e quindi può essere visionato.

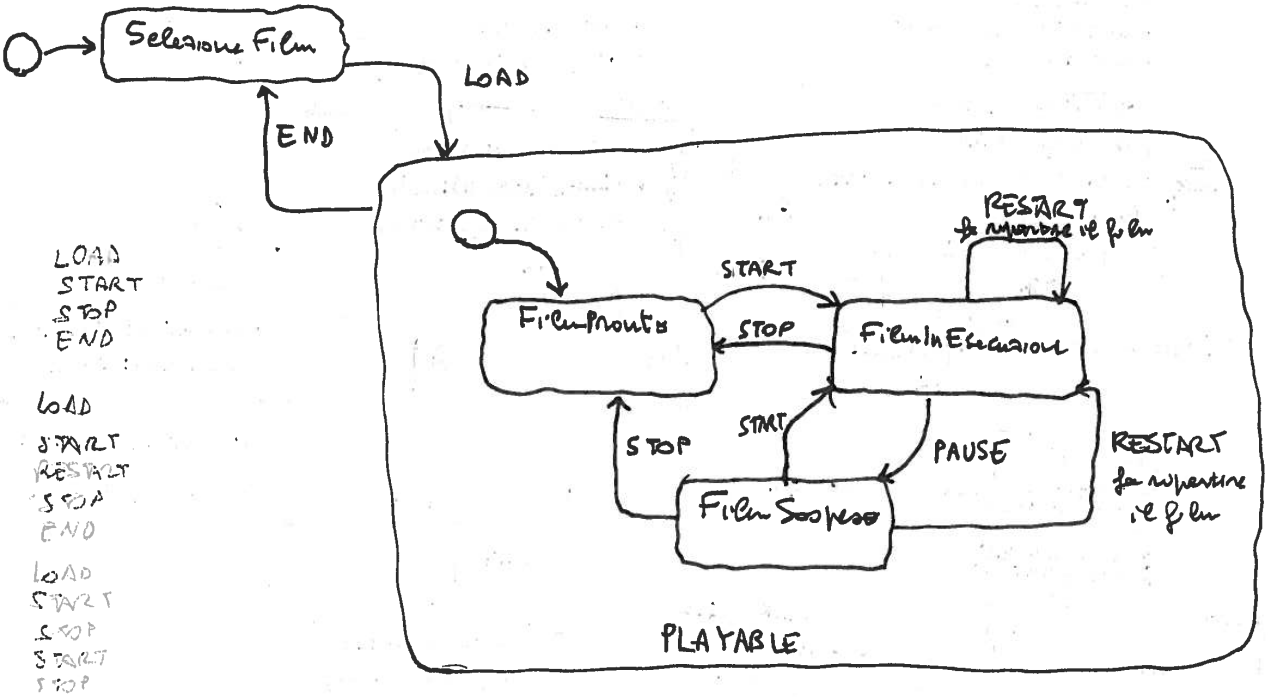
**Soluzione**



2) Caso d'uso utente che si autentica, sceglie di recensire un film e poi sceglie di recensire un artista




3)




LOAD
START
STOP
END

LOAD
START
RESTART
STOP
END

LOAD
START
STOP
START
STOP
END

LOAD
START
END

# Esercizio 2

Si consideri una variante del problema dei film di cui all'Esercizio 1, in cui esistono solo i film e i film muti. Si considerino in Java le classi **Film** e **FilmMuto** e si discuta se **Film** debba essere sottoclasse di **FilmMuto** o viceversa nei seguenti casi alternativi:

1. **Film** ha due metodi **startPlayingSound()** e **startPlayingVideo()** mentre **FilmMuto** ha solo il metodo **startPlayingVideo()**.

2. entrambe le classi hanno un solo metodo **startPlaying()** e inoltre esistono due metodi osservatori **hasVideo()** e **hasAudio()**. Il metodo **startPlaying()** ha precondizione TRUE per entrambe le classi. Per la classe **Film** ha post-condizione **hasVideo() && hasAudio()**, mentre per la classe **FilmMuto** ha post-condizione **hasVideo()**.

3. come sopra, ma la post-condizione di **startPlaying()** di **FilmMuto**, è **hasVideo() && !hasAudio()**.

**Soluzione**

1. **Film** può ereditare da **FilmMuto**, mentre il viceversa violerebbe la regola della segnatura.

2. **Film** può ereditare da **FilmMuto**, mentre il viceversa violerebbe la regola dei metodi, in quanto la postcondizione di **Film.startPlaying()** implica la postcondizione di **FilmMuto.startPlaying()** (ossia è pi forte), ma non viceversa.

3. Nessuna possibilità di fare ereditare una dall'altra, in quanto la postcondizione di **Film.startPlaying()** e quella di **FilmMuto.startPlaying()** sono in contraddizione: nessuna delle due implica l'altra.

# Esercizio 3

Si supponga di voler realizzare un'applicazione per gestire i film di una videoteca personale. Sono date le classi **Film**, e **Artista**, che contengono le ovvie informazioni: titolo, anno, regista, lista di attori per **Film**, nome cognome, etc per **Artista**. La classe **Videoteca** è una collezione di **Film**.

Si definisca e si implementi in Iava per **Videoteca** un iteratore **direttoDa(Artista a)** che consente di iterare su tutti i film della collezione diretti da un certo regista.

Il metodo **bestOf(Artista a)** restituisce il film con la migliore recensione di un certo regista passato come parametro. Fornirne la specifica JML. La specifica può basarsi su opportuni metodi osservatori di Videoteca e sulla disponibilità di un metodo osservatore **getValutazione()** per avere la valutazione di un certo film. La valutazione è un intero da 1 (min) a 5 (max), ipotizzando che una mancata recensione di un film di un regista valga 0.

Si considerino i seguenti due casi per gestire la situazione di "regista non presente nella collezione":

1. lo esclude la precondizione;
2. non lo richiede la precondizione, ma, nel caso, viene generata un'opportuna eccezione.

**Soluzione** All'interno della classe Videoteca si definiscono il metodo e la classe privata per l'iteratore:

```
public class Videoteca {
  private ArrayList<Film> film;
  //@ensures (*\result e' true se f e' nella collezione *);
  public /*@ pure @*/ boolean contains (Film f);
...
  public  direttoDa(Artista a) {
    return new VideoIter(this, a);
  }
  private static class VideoIter implements Iterator<Film>  {
    private Videoteca vid;
    private Artista reg;
    private int n;
    private /*@ helper @*/ searchNext() {
      for(; n<vid.size() && !vid.film.get(n).regista().equals(reg); n++);
    }
    VideoIter(Videoteca v, Artista a) {vid=v; reg=a; n=0; searchNext();}
    public boolean hasNext{return n<vid.size();}
    public Film next() throws NoSuchElementException {
      if (n>=vid.size()) throw new NoSuchElementException();
      Film f = film.get(n);
      searchNext();
      return f;
    }
}
```

Specifica del metodo **bestOf(Artista a)**: Si usa il metodo puro contains definito sopra per Videoteca. Versione con precondizione:

```
//@ requires (\exists Film f; this.contains(f); f.regista().equals(a));
//@ ensures this.contains(\result) && \result.regista().equals(a) &&
//  il film restituito ha valutazione massima:
//@ (\forall Film f; contains(f) && f.regista().equals(a);
//@        \result.getValut(a)>= f.getValut(a));
public Film bestOf(Artista a)
```

Versione con eccezione:

```
//@ ensures this.contains(\result) && \result.regista().equals(a) &&
//@ (\forall Film f; contains(f) && f.regista().equals(a);
//@         \result.getValut(a)>= f.getValut(a));
```

```
//@ signals(NotFoundException e)
//@        !(\exists Film f; contains(f); f.regista().equals(a));
public Film bestOf(Artista a) throws NotFoundException;
```

# Esercizio 4

1. Si consideri la specifica fornita nell'Esercizio 1 (parte 3) per la classe **Film**, che descrive come poter vedere un film. In base a questa descrizione, quali sequenze di invocazioni di metodi della classe potrebbero essere definite per testare la classe stessa secondo un metodo black-box? Per ciascuna sequenza di invocazioni, si dica quale è lo scopo atteso del test.

2. Si consideri il seguente metodo in Java che effettua la ricerca binaria in un array:

```
static int binarySearch(int[] search, int find) {
   int start, end, midPt;

   start = 0;
   end = search.length - 1;
   while (start <= end) {
      midPt = (start + end) / 2;
      if (search[midPt] == find) {
         return midPt;
      }
      else if (search[midPt] < find) {
         start = midPt + 1;
      } else {
         end = midPt - 1;
      }
   }
   return -1;
}
```

   Si fornisca un insieme di casi di test minimo che consenta la copertura di tutte le istruzioni del programma.

**Soluzione**

1. Alcune sequenze significative:
   LOAD START STOP END uscita corretta in caso di stop
   LOAD START STOP START il film riparte dall'inizio
   LOAD START PAUSE START il film riprende dal punto di interruzione (pausa)
   LOAD START RESTART il film riparte dall'inizio
   LOAD START PAUSE RESTART il film riparte dall'inzio dopo una pausa

2. Bastano due casi di test.

   a) `search = [1], find = 0;`
   b) `search = [0,1,2,3,4,5,6], find = 1`

   Il caso (a) copre l'istruzione return -1. Il caso (b) fa percorrere tutte le altre istruzioni.